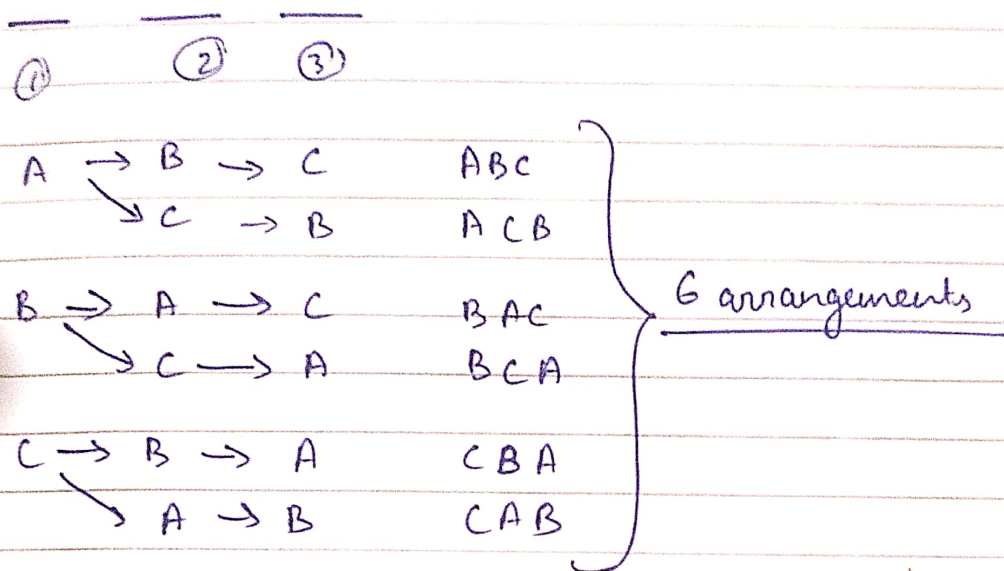


Backtracking

Suppose we need to arrange ³ students in a line
All possible arrangements



Total permutations = $n!$

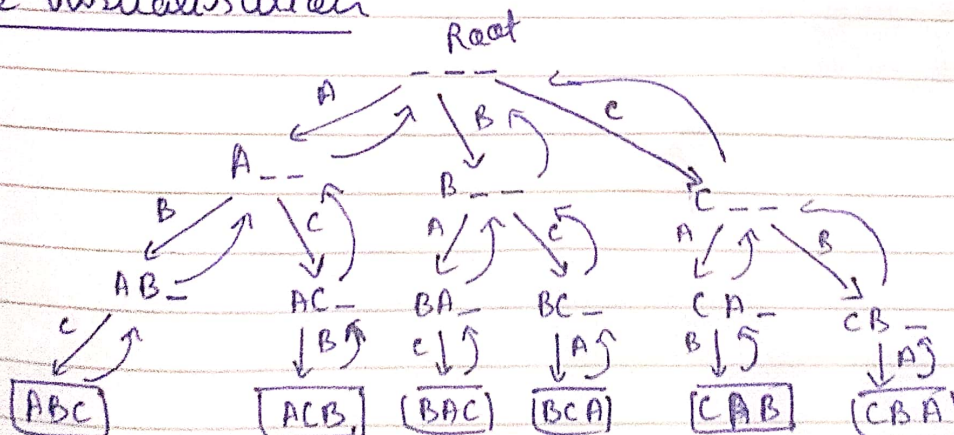
So $3! = 3 \times 2 = 6$ ways.

If arranged in 4

then $4! = 4 \times 3 \times 2 = 24$ ways.

etc:

Tree visualisation



Day 9

camlin
Date: / /

Subset backtracking

Backtracking is postponed to later days.

- 1) class → Properties → Functions
- 2) object → call in main

```
class pen {
```

```
    String color;
```

```
    String type;
```

```
    public void write() {
```

```
        System.out.println("write something");
    }
```

```
    public void printColor() {
```

```
        System.out.println(this.color);
    }
```

```
}
```

```
public class OOPS {
```

```
    public static void main(String args[]) {
```

```
        pen pen1 = new pen();
```

```
        pen1.color = "blue";
```

```
        pen1.type = "gel";
```

```
        pen pen2 = new pen();
```

```
        pen2.color = "red";
```

```
        pen2.type = "ball";
```

```
        pen1.write();
```

```
}
```

```
}
```

- // classes names starts with a capital letter
- // functions names starts with a small letter.

Day 9

```
class Student {
```

```
    String name;  
    int age;
```

```
    public void studentInfo() {  
        sout (this.name);  
        Sout (this.age);  
    }
```

①

Default constructor

```
public class COPS {
```

```
    privm (String args[]) {
```

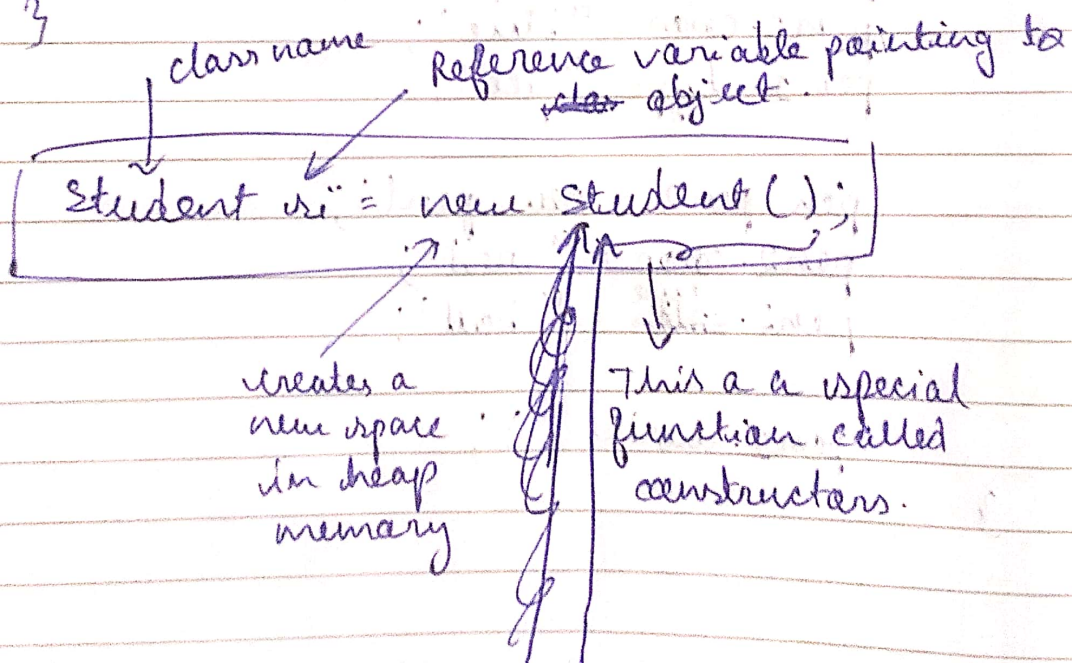
```
        Student si = new Student();
```

```
        si.name = "Sharat";
```

```
        si.age = 20;
```

```
        si.studentInfo();  
    }
```

3



Constructor
(special function)

Constructor do not return anything
// constructor can be called once for an object.

Inside class.

```
class Student {
    String name;
    int age;
```

This here is a ~~void~~ ^{no} constructor. so java will stop making a default constructor.

```
    Student () {
        cout << "constructor called";
    }
```

②

No Argument constructor

```
    Student (String name, int age) {
        this.name = name;
        this.age = age;
    }
```

This will initialise the value of name and age at the calling stage

③

Parameterised constructor

so in main

```
Student s1 = new Student("aman", 24);
s1.studentInfo();
```

④

copy constructor

```
Student (Student s2) {
    this.name = s2.name;
    this.age = s2.age;
}

Student () {}
```

In main

```
Student s1 = new Student();
s1.name = "aman";
s1.age = 24;

Student s2 = new Student(s1);
s2.printInfo();
```


Day 9

→ destructors / garbage collectors
so Java does garbage collection automatically if something is not being used.

Polymorphism (compile time)
many forms.

For eg:

```
class student {
```

```
    String name;
```

```
    int age;
```

```
    public void displayInfo (String name) {  
        cout (name);  
    }
```

```
    public void displayInfo (int age) {  
        cout (age);  
    }
```

```
    public void displayInfo (String name, int age) {  
        cout (name);  
        cout (age);  
    }
```

Method
overloading
(same name function
with different
functionality)

This is called
compile time polymorphism

Day 9

camlin

Date : _____

Polymorphism (Runtime)

Polymorphism is the ability of a variable to take on many forms. In runtime polymorphism, the method to be executed is determined at runtime.